INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	**TECNOLOGÍA DE OBJETOS**				
TÍTULO DE LA PRÁCTICA:	*C++ Funciones lambda, introducción a Qt / Widgets*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026-B*	**NRO. SEMESTRE:**	
FECHA DE PRESENTACIÓN:	*21/10/2026*	**HORA DE PRESENTACIÓN:**	*23:59*		
INTEGRANTE (s): *-Ccahua Chise Elizabeth Karina* *-Navarro Quispe Henry Alex* *-Davila Flores Mathias Dario*				**NOTA:**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. PROBLEMAS PROPUESTOS RESUELTOS:

Repositorio: https://github.com/mathiasddf/Lab_04_TO

1. *Contar los números primos y menores de 1000 de un vector de números enteros generados de forma aleatoria (la solución puede usar el método std :: count_if)*

- **Código**

```C/C++
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

int main() {
    std::vector<int> numeros;
    std::mt19937 generador(std::random_device{}());
    std::uniform_int_distribution<int> distribucion(1, 1500);

    for (int i = 0; i < 30; ++i) {
        numeros.push_back(distribucion(generador));
    }
```

```
    auto esPrimo = [](int numero) {
        if (numero < 2) {
            return false;
        }

        for (int divisor = 2; divisor * divisor <= numero; ++divisor) {
            if (numero % divisor == 0) {
                return false;
            }
        }

        return true;
    };

    int cantidad = std::count_if(
        numeros.begin(),
        numeros.end(),
        [&esPrimo](int numero) {
            return numero < 1000 && esPrimo(numero);
        }
    );

    std::cout << "Numeros generados: ";
    for (int numero : numeros) {
        std::cout << numero << " ";
    }

    std::cout << "\nCantidad de primos menores que 1000: "
              << cantidad << "\n";

    return 0;
}
```

- **Explicación de los elementos principales**
  - **Vector números:** almacena los enteros aleatorios generados.
  - **Generador y distribución:** std::mt19937 genera los valores y std::uniform_int_distribution limita el rango entre 1 y 1500.
  - **Lambda esPrimo:** comprueba si un entero es primo. Los valores menores que 2 no son primos.
  - **std::count_if:** recorre el vector y cuenta los valores para los que la condición devuelve verdadero.
  - **Captura [&esPrimo]:** permite que la lambda usada por count_if acceda a esPrimo.

- **Compilacion y Ejecucion**

```
sh-5.3$ g++ ejercicio01.cpp -o ejercicio01
sh-5.3$ ./ejercicio01
Numeros generados: 1393 140 1335 449 100 999 1343 739 1402 204 1025 132 692 78 1086 503 824 73 958 808 729 712 1437 1464 1218 1420 1468 1171 1206 183
Cantidad de primos menores que 1000: 4
sh-5.3$
```

- **Explicación de Resultados**

  El programa generó un vector de números enteros aleatorios y contó cuántos de ellos eran números primos menores que 1000. Para realizar el conteo se utilizó std::count_if, que revisa cada elemento del vector y verifica si cumple la condición establecida. En esta ejecución se encontraron ___ **números primos menores que 1000**. Al ejecutar el programa nuevamente, el resultado puede cambiar porque los números se generan aleatoriamente.

2. ***Crear una interface gráfica (que implemente señales y slots) que muestre una lista de países, al dar clic sobre alguno que se muestre un Label o Text con el idioma y capital correspondiente.***

- **Implementación de señales y slots**

  La señal itemClicked del objeto QListWidget se conecta con el slot mostrarInformacion de la clase MainWindow:

```C/C++
connect(
    ui->listaPaises,
    &QListWidget::itemClicked,
    this,
    &MainWindow::mostrarInformacion
);
```

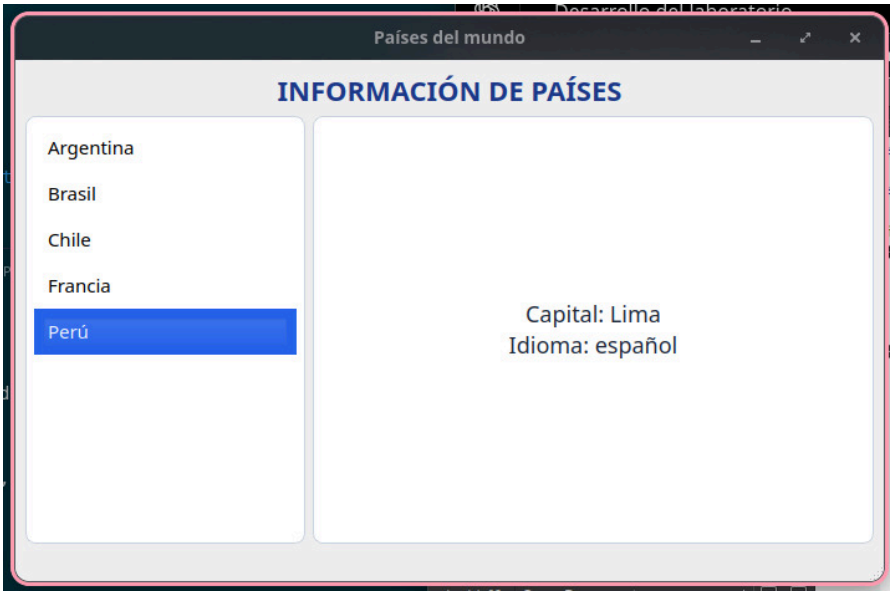
  Cuando el usuario hace clic en un país, se activa la señal itemClicked. La conexión ejecuta el slot mostrarInformacion, que consulta los datos del país y actualiza la etiqueta:

```C/C++
void MainWindow::mostrarInformacion(QListWidgetItem *item) {
    ui->informacion->setText(datosPaises.value(item->text()));
}
```

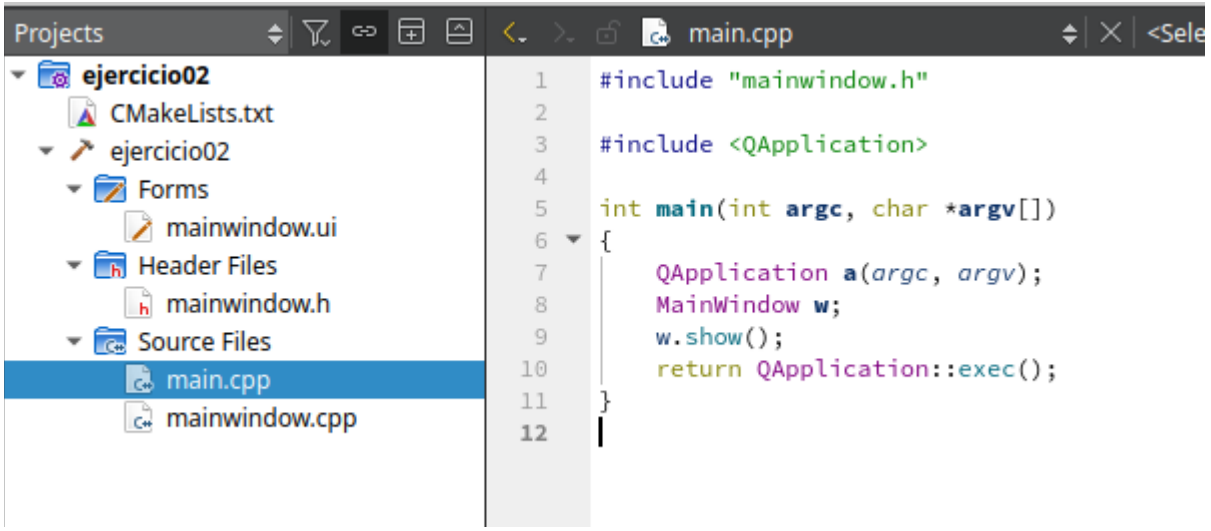
En esta implementación, la señal comunica que ocurrió un clic y el slot define qué acción realizar como respuesta.

- **Prueba y resultados**

  Se ejecutó la aplicación y se seleccionaron países de la lista. La interfaz mostró la capital y el idioma correspondientes. Por ejemplo, al seleccionar Perú, apareció "Capital: Lima" e "Idioma: español". Esto confirma que la señal y el slot están conectados correctamente.



- **Árbol de archivos en Qt Creator**



II. CUESTIONARIO:

Buscar las funciones lambdas que se utilizan y/o son soportadas en compiladores C++ (versiones).

Las funciones lambda fueron incorporadas en C++11 y cada estándar posterior les agregó mejoras. Pero que el estándar exista no significa que el compilador (GCC, Clang, MSVC) ya lo tenga. Cada compilador va "poniéndose al día" en versiones distintas.

Estándar	Novedad en lambdas	Funcion
C++11	Lambdas básicas: capturas [=], [&], [this], parámetros y tipo de retorno	Escribir funciones cortas sin declararlas aparte, por ejemplo junto a for_each
C++14	Lambdas genéricas (parámetros con auto) y capturas con inicialización	Una misma lambda trabaja con varios tipos de datos
C++17	Lambdas constexpr implícitas y captura de *this por valor	Usar lambdas en cálculos de compilación y evitar problemas con objetos que se destruyen
C++20	Lista explícita de plantillas, expansión de paquetes en capturas, lambdas sin estado construibles por defecto	Más control sobre los tipos y mayor compatibilidad con plantillas
C++23	Atributos en lambdas y operador de llamada estático	Mejor optimización y mayor flexibilidad

Desde qué compilador funcionan:

Se muestra desde qué versión de cada compilador se puede usar una característica de las lambdas. Cada fila es una mejora que salió en una versión de C++, y las columnas de GCC, Clang y MSVC indican la primera versión de ese compilador que ya la acepta. Si se usa una versión más antigua, el código con esa característica da error al compilar.

Lambda	Estándar	GCC	Clang	MSVC
Lambda básica	C++11	4.5	3.1	10 (VS 2010)
Lambda genérica (auto)	C++14	4.9	3.4	19.0 (VS 2015)
Captura de *this	C++17	7.0	3.9	15.3 (VS 2017)

Ejemplo: Este código se compila con la opción -std=c++14 o superior.

C/C++

```
// C++11: lambda básica que suma 3
auto sumar3 = [](int x) { return x + 3; };
cout << sumar3(5) << endl;              // 8

// C++14: lambda genérica, sirve para int y double
auto doble = [](auto x) { return x * 2; };
cout << doble(4) << " " << doble(2.5) << endl;   // 8 y 5
```

CONCLUSIONES

- *Comprendí que un estándar exista no significa que el compilador ya lo tenga. Por ejemplo, las lambdas básicas funcionan desde GCC 4.5 y Clang 3.1, pero las genéricas necesitan GCC 4.9 o Clang 3.4. Por eso, antes de usar una característica nueva de lambda, conviene revisar la versión del compilador y del estándar con el que se está trabajando.*

METODOLOGÍA DE TRABAJO

REFERENCIAS Y BIBLIOGRAFÍA

[1] B. Filipek, “Lambdas: From C++11 to C++20, Part 2”: https://cppstories.com/2019/03/lambdas-story-part2

[2] cppreference, “C++ compiler support”: https://en.cppreference.com/w/cpp/compiler_support